

INDEX

Numbers

010 Editor, 166
32-bit architecture, 124–125
3D meshes, 20, 23, 126, 137
64-bit architecture, 112, 124–125

A

abstraction layers, 78, 116, 130, 138
 file access, 24
 hardware, 130–132
 randomness sources behind, 72, 74
accessibility, 5
access violations, 60, 91, 94
ack search tool, 86
Activity Monitor, 173
AddressSanitizer, 169
AFL++, 166
ag search tool, 86
AI (artificial intelligence), 31, 71
algorithms, 28, 116, 138, 207, 208
altWinDirStat, 172
amalgamated builds, 20
AMD Radeon GPU Profiler, 170
APIs, 28
 asynchronous IO, 28
 foundational, 41
 graphics, 130, 131, 170
 implementation patterns, 42
AppData directory, 89
AR (augmented reality), 156
archives, keeping, 145
artificial intelligence (AI), 31, 71
ASCII, 5, 40, 89
assertions, 63–65, 67, 92. *See also*
 instrumenting programs
 adding, 63–64
 commenting out, 204
 enabling and disabling, 69
 vs. error handling, 64
 information from, 64–65
 as programmatic breakpoints,
 61, 104
 as safeguards, 189
 static, 65
 triggering crash dumps, 92
 verifying data with, 127
`assert()` macro, 63
assets, 20, 23, 122
assignment request, xxi
attrition, preparing for, 198–199
augmented reality (AR), 156
automated tests, 190, 207, 211–212
automating the reproduction, 29–35
 hardcoding steps, 30–31
 immediate reproduction,
 33–34
 recording inputs, 31–32
 scaling, 30
 with scripts, 31
 specific implementations,
 targeting, 32–33
 as test cases, 190
 when it isn't possible, 34–35
 whether to automate, 29–30

B

backporting, 52, 57–58
Backtrace, 97
bad input, 66
bad pointers, 7, 46, 213
Bazel, 20
Beyond Compare, 165
binary builds, testing with, 55
binary data, 107, 166

- binary files, 87, 89–90, 165–166, 185, 202–203
- binary search, 54, 56
- Bing, 49
- bit flips, 79
- “black box” software, 43, 173
- Black Duck, 168
- blame function, 193
- Boost.ASIO, 78
- Boost.Log, 162
- boxing and unboxing (C#), 121
- branches, 8, 52, 58–59, 190
- Breakpad, 96
- breakpoints, 39, 59, 100, 103, 107–108
 - data, 112, 126
 - placement strategy, 61, 104–105
 - removing and disabling, 61, 108
 - tracking data changes with, 126
 - triggering, 64–65
 - on Windows, 65
- break statement, 60
- browser developer tools, 169
- bug fixes
 - collaborating on, 185
 - committing one at a time, 181–182
 - criteria for, 180–181
 - documenting, 143, 186–187
 - early, 196–201
 - quick, 201–206
 - reversible, 181
 - root cause, addressing, 181–183
 - side effects, testing for, 185–186
 - verifying, 5, 182
- bug reports, 11–15, 53, 83–84
 - example, 13–14
 - contents of, 12–13, 16
- bugs
 - critical, 197, 214
 - definition of, xviii
 - reasons to fix, xviii–xix
 - taking responsibility for, xx–xxi, 200–201
- bug trackers, xxi, 5, 14, 49, 52
- bug triage, 84, 219
- build configurations, 100–104
- build times, reducing, 19–23
- bulk changes, 195
- bus factor, 199

C

- C, 7, 19
 - assertions in, 63, 65
 - build times, 19–20
 - crash dumps, 93
 - exception handling, 60
 - hot reloading, 25
 - network-related functions, 78
 - PRNG-related functions, 76
 - string formatting, 161
 - time-related functions, 73
 - unsafe string functions, 136
- C++, 7, 19
 - assertions in, 63, 65
 - auto keyword, 116
 - build times, 19–22
 - compilers, 103
 - configuration options, 87
 - exception handling, 60
 - features to look out for, 118–120
 - hot reloading, 25
 - logging libraries, 162
 - macros, 120
 - modules (C++20), 22
 - network functions, 78
 - PRNG-related functions, 76
 - strings, 119
 - templates, 22
 - threading-related functions, 75
 - time-related functions, 65
 - undefined behavior, 103
- C#, 19
 - assertions in, 63
 - boxing and unboxing, 121
 - build times, 19–20
 - exception handling, 60
 - hot reloading, 25
 - features to look out for, 120–121
 - logging libraries, 162
 - network-related functions, 78
 - pitfalls, 120–121
 - PRNG-related functions, 76
 - threading-related functions, 75
 - time-related functions, 73
- call sites, searching for, 195
- call stacks, 60–61, 92, 95, 105–107
- Ccache, 20

- changelogs, 56, 130
- changes, implementing, 183–185
- character encodings, 89, 124, 127
- chat services, 49, 82, 145–146
 - as debugging logs, 145–146
- cheat codes, 151
- Clang, 21, 53, 101–102
 - disabling optimizations in, 53, 204
 - sanitizers, 168–169
 - warnings, 168
- clean room implementation, 208
- clean shutdowns, 7
- CMake, 20
- code comments, 44, 187
- Code Complete* (McConnell), 225
- code modifications, 134–135
- CodePen, 167
- code playgrounds, 167
- code sanitizers, 168–169
- coding standards, 191
- command line arguments, 30, 46, 110, 150, 173
- comma-separated values (CSV)
 - files, 143
- comments
 - as clue to difficult systems, 44
 - commenting out code, 138, 155–156, 202, 204, 216
 - on disabled code, 217
 - documenting fixes in, 143, 187
 - readable code and, 116
 - searchable markers in, 22, 90–91, 140, 177, 206
 - searching for similar, 194
- commit messages, 54–56, 186–187, 193–194, 206
- commits, 54–56, 188, 193–194
 - identifying the offending change, 56
- Compiler Explorer, 167
- compile times, avoiding long, 22
- compression, 24, 28, 122
 - algorithms, 28
 - of symbol files, 95
 - texture, 23
- concurrency, 74, 107
- conditional breakpoints, 104–105
- conditional statements, 61
- configuration files, 20, 30, 89, 125
- Confluence, 145
- connection speed, fixing, 24
- continue statement, 60
- continuous integration system, 55, 168, 224
- control flow, 59–60, 67, 102
 - statements, 60
- Control Flow Integrity, 169
- cosmic radiation, 79
- Coverity, 168
- Cppcheck, 168
- CPUs, 28, 95, 96, 104, 154
 - architectures, 110
 - core, 23, 71, 74
- crash dumps, 13, 30, 38, 91–97
 - choosing the right type, 95
 - gathering data from, 91–92, 95
 - implementing, 92–94
 - with C, 93
 - out-of-process handler, 94
 - triggering, 91–92
- Crashpad, 96
- creative solutions, finding, 212–219
- curl, 38, 78
- custom-built debuggers, 62
- custom tools, 146–152
 - implementing, 147
 - for libraries, 150
 - recommended, 149, 152
 - release considerations, 152
- curl, 38–39, 78

D

- dangling pointers, 119
- data, 122
 - finding bad data, 126–127
 - good vs. bad, 127
 - original input, 123
 - permanent vs. transient, 125
 - symptoms of bad data, 123–126
 - transformation of, 122
- DataFlowSanitizer, 169
- data structures, 130
- DB Fiddle, 167
- deadlocks, 43, 130
- dead zones, 200
- Dear ImGui, 147

- Debug.Assert() method, 63
- debug builds, 22, 68, 100, 103–104
- debuggers
 - attaching, 59
 - memory view, 68, 107, 125
 - when not to use, 62, 159–160
- debugging
 - advanced, 111–113
 - inevitability of, xix
 - logs
 - archiving, 146
 - chats as, 145–146
 - documentation tools for, 144–145
 - keeping, 140–146
 - summarizing findings, 143
 - tools, 144–145
 - tracking progress in, 141–142
 - rewards of, 221
 - starting the process, 104–110
 - workflow, 1–4
 - examine step, 3–4
 - fix step, 4–5
 - iterative nature of, 2, 99, 206, 222
 - probe step, 3
 - reproduce step, 2
- DebugView, 172
- decoupling, 45
- delta times, 71–73
- denormalized numbers, 124
- dependencies, xix, 8, 57, 172, 215
 - chains, 45
 - data, 4
 - eliminating, 24
 - tracking, 23, 25
- Dependencies tool, 172
- Dependency Walker, 172
- dependent behavior, avoiding, 199–200
- deployment, 8, 23–24
- deserialization, 28
 - zero-copy, 28
- determinism, xxiii, 70–72
- diff tool, 127, 165
- Direct3D, 41, 131
- DirectStorage, 28
- dirty room implementation, 209–210
- disabling code, 155–156, 215–217
 - commenting out, 216
 - at compile time, 190, 216
 - at runtime, 216–217
- disassemblers, 165–166
- Discord, 49
- distcc, 20
- distractions, avoiding, 143–144
- distributed caching, 20
- distributed compilation, 20
- division by zero, 60, 91, 120, 124
- D language, 65
- dmalloc, 171
- dmesg, 172
- documentation
 - of bug fixes, 143, 186–187
 - as clue to difficult systems, 44
 - of debugging tools, 148, 152
 - of decisions, 218
 - of known issues, 133, 141, 219
 - searching, 52, 89, 133
 - of specifications, for rewrites, 208–209
 - tools for, 144–145
 - of workarounds, 206
- dogfooding, 214
- dot directory, 89
- drivers, 130–132
- dry runs, performing, 149
- DTrace, 172
- DuckDuckGo, 49
- duck test, 128
- dynamic link libraries, 25, 94, 172

E

- Electron, 41, 83
- Elements of Programming Style, The* (Kernighan and Plauger), xx
- EMacs, 167
- entry points, 3, 37, 97
- enums, 67
- environment changes, 205
- error codes, 66–67
- error-correcting codes (ECCs), 79
- error handling, 64, 65–67, 69, 191
- error messages, searching for, 50
- error signaling, 67

- error value, 66
- Everything (search tool), 87, 166
- exceptions, 60, 67, 118
 - as control-flow statements, 60
 - breaking on, in debuggers, 60
 - hardware, 91–92
 - working around, 204
- execution pointers, 103
- exit codes, 66
- expectations, managing, xxii
- explicit type conversions, 118
- external tools, 148–149. *See also*
 - custom tools
- extern template, 22

F

- FASTBuild, 20
- fast-math option, 71
- features disguised as bugs, 184
- feedback loop, 19
- Feynman technique, 173
- Fiddler, 170
- file access functions, 46
- FileLocator Pro, 86
- file locks, 8, 43, 172
- File Locksmith, 172
- filepaths, 33, 40, 87
- file search tools, 87, 166
- Firebase Crashlytics, 97
- fixed time steps, 71–72
- floating-point inaccuracies, 7–9
- floating-point numbers, 7, 32
 - delta times, 71–73
 - determinism and, 71
 - inaccuracies, 7–8, 71, 184–185
 - special values (IEEE 754), 124
- foreign language fonts, 155
- forks, 58
- format linters, 166
- format validators, 166
- forums, 49, 82
- forward declaration, 21
- frameworks, widely used, 41
- frequently asked questions list, 82
- full dumps, 95
- full-text search, 86, 145
- function calls, 4, 33, 112
- function parameter, 106, 193

- future-proofing, 181
- fuzzers, 166
- fuzzing, definition of, 166
- fuzzy search, 87
- fzf, 87

G

- games
 - AI-driven reproduction in, 31
 - assets, 20, 23–25
 - debugging commands for, 151–152
 - determinism in, 71
 - saved games as shortcuts, 34
- garbage collection (C#), 121
- GCC, 21, 168, 169
- GDB, 113
- Ghidra, 166
- git bisect command, 54
- Gitflow, 58
- GitHub, 51, 57, 83
- git revert command, 202
- global variables, 22, 107
- Godbolt, 167
- god mode, 151
- Go language, 52, 113
- goto statement, 60
- graphics APIs, 41, 130–131, 170
 - debugging tools for, 170–171
 - validation layers, 170–171
- graphics cards, 85, 96, 170
- graphics processing units (GPUs), 170
- graphics tools, 170–171
- grep, 86
- guard values, 67
- GUI, 18, 20
- gyroscopes, 32, 205

H

- Hacker News, 49
- handle_exception() function, 93
- hardware abstraction layers, 130
- hardware issues, 37, 79, 132–133, 205
- header files, 20, 22
- help, asking for, 79–85
- hexadecimal, 68
- hex editors, 90, 166, 203
- Honggfuzz, 166
- hot reloading, 25–26

HTTP requests, 24, 115, 138
Hunt, Andrew, 225
HxD, 165
hybrid tools, 149. *See also* custom tools
hypotheses, 134
 confirming or rejecting, 137
 examples, 138–139
 formulating, 135
 testing, 135

I

IDA Pro, 166
IDE (integrated development environment), 38, 61, 86, 102, 103
IEEE standard, 71, 124
if statement, 59, 61, 64
immediate mode GUI (IMGUI), 147
immediate reproduction, 33
implicit type conversions, 118
improving your code. *See* readable code
#include directive, 20
include-what-you-use, 21
Incredibuild, 20
indirection, xxiii, 3, 8
infinite loops, 43, 141
infinite recursion, 106
InfluxDB, 41
inlining, 102
input events, 31–32, 43
 recording, 31–32
integrated development environment (IDE), 38, 61, 86, 102, 103
integrated tools, 147–148. *See also* custom tools
interdependencies, 18, 188
internet searches, 47–52
inter-process communication, 77
intranets, 52
invalid memory access, 119
IP addresses, 163
irrelevant systems, 45–47
ISO standard, xviii
iteration times, 16, 33, 135, 155, 207, 211

J

Java, 19, 113, 115, 132, 168
JavaScript, 20, 115, 167, 169

JSFiddle, 167
JSON, 124
jumbo builds, 20

K

KDiff3, 165
Kernighan, Brian W., xx
known issues lists, 133, 141, 219

L

launchers, 110
LeakSanitizer, 169
libFuzzer, 166
LinkedIn, 85
link-time optimization (LTO), 22
Linux, 73, 113
Live++, 25
LLDB, 113
local variables, 106
log files, 30
logging libraries, 162–163
logs, tools for, 172
log statements, 68, 158
Loguru, 162
Lua, 20, 115, 160

M

magic numbers, 67–68, 116
magic values, 67–68
Maguire, Steve, 67
main() function, 33, 46, 61, 93, 114, 139
make tool, 110
markdown-formatted text, 144
masking bug symptoms, 180, 203, 205, 213
Mastodon, 49
Mathematica, 117
matrix multiplication, 135
McConnell, Steve, 225
MediaWiki, 145
member variables, 106–107
Memoirs of a Computer Pioneer (Wilkes), xix
memory
 address, 7, 112
 allocation, 74
 blocks, 107
 corruption, 46, 68–69, 103, 107

- leaks, 119, 169, 171–172
- mapping, 28
- tools, 171–172
- uninitialized, 68, 73–74, 103, 169
- view, in debuggers, 68, 107, 125
- memory_profiler, 171
- MemorySanitizer, 169
- menus, removing at startup, 27
- Metal (graphics API), 52, 131, 170
- Microsoft Docs, 49
- Microsoft Teams, 145
- middleware, 48, 57, 148, 155
- mini dumps, 95
- MiniDumpWriteDump() function, 95
- minimal repros, building, 153
- mmap() function, 28
- mocking, 115
- model-view-controller (MVC)
 - pattern, 42
- MongoDB, 41
- Morrowind*, 204
- Mozilla Developer Network, 49
- MSVC, 21, 168
- multitasking, 74
- multithreading, 28, 33, 41, 44, 69, 71, 75, 107, 204
- mutexes, 43, 75, 204
- MySQL, 41

N

- NaN (Not a Number), 124
- Natvis file, 111
- negative infinity, 124
- networking
 - complexity, 18–19
 - functions, 78
 - tools, 169–170
- NLog, 162
- nondisclosure agreements, 83, 174
- nontext files, 89
- Not a Number (NaN), 124
- Notepad++, 167
- Notion, 145
- Nsight Aftermath, 95
- Nsight Graphics, 153, 170
- null pointers, 7, 64, 91, 109, 114, 124
- Nvidia, 85, 95, 153–154, 170

O

- objdump, 166
- OBS Studio, 166
- off-by-one error, 7
- off-the-shelf tools, 164–173
 - availability of, 165
 - code playgrounds, 167
 - code sanitizers, 168–169
 - diff and merge tools, 165
 - disassemblers, 165–166
 - general-purpose, 165–167
 - graphics tools, 170–171
 - memory tools, 171–172
 - networking tools, 169–170
 - process tools, 173
 - static analyzers, 167–168
 - system tools, 172–173
 - when to reach for, 165
- Oodle, 28
- OpenGL, 85, 131
- open source software, 20, 57–58, 77, 96, 130, 165
- operating systems
 - as environment for bugs, 132–133
 - input recording at OS level, 32
 - scheduler, 71, 74
 - tracing tools, 172
- operator overloading, 118
- optimization, effects of, 100–103
- optimization levels, 22, 102
- Overlapped IO, 28
- overlying assets, 24
- ownership, taking, xx–xxi, 200–201

P

- P4Merge, 165
- pair programming, 208
- passwords, 157, 163
- .pdb* format, 94
- Perl, 113
- persistent data, 199
- personal data, xix, 157
- PHP, 20
- PIMPL (pointer-to-implementation)
 - idiom, 21
- PIX, 170
- plaintext files, 144

- Plauger, P.J., xx
- pointers, 7, 74, 107, 119, 124–125
- pointer-to-implementation (PIMPL)
 - idiom, 21
- polymorphism, 118
- POSIX, 75, 78
- positive infinity, 124
- PostgreSQL, 41
- Postman, 170
- postmortem meetings, 189
- Pragmatic Programmer, The* (Thomas and Hunt), 225
- precompiled header files, 20
- pre-conditions and post-conditions, 41,
 - 44, 63, 192–193
- preprocessor commands, 216
- presentations, preparing, 177
- printf, 158–164
 - cleaning up, 163–164
 - implementing, 160–162
 - logging, 162–163
 - pitfalls, 159
 - statements, 158
 - when to use, 159
- privacy, xix, 163
- PRNGs, 70, 76, 77
- Process Explorer, 110, 172, 173
- Process Hacker, 173
- Process Monitor, 172
- process tools, 173
- programming languages, 117–122
- project history, checking, 52
- project management tools, xxi, 52, 89,
 - 143, 187, 194, 200, 219
- ps command, 110, 173
- pseudo-random number generators
 - (PRNGs), 70, 76, 77
- Pthread, 75
- PVRTexTool, 171
- PVS-Studio, 168
- Pylint, 168
- Python
 - build times, 20
 - features to look out for, 121–122
 - hot reloading, 25
 - network-related functions, 78
 - PRNG-related functions, 76
 - threading-related functions, 75

- time-related functions, 73
- versions, 122

Q

- quality assurance (QA) teams, xxi, 5–6,
 - 12, 185–186, 195–196,
 - 214, 224
- Qt, 83
- Questioning everything, 127–134
- quick fixes, deploying, 201–206
- Quill, 162

R

- race conditions, 46, 62, 169
- RaiseException() function, 91
- RAM, 79
- rand() function, 47, 76
- randomness, 69–79
- random number generators, 75. *See also*
 - pseudo-random number generators (PRNGs)
- raw data, inspecting, 171
- React, 41
- readable code, principles of, 115–117
- reading code, 113–122
- read-only files, 8, 43
- Reddit, 49, 83
- refactoring source code, 183
- Regex101, 88
- RegExr, 88
- regressions, preventing, 158, 190
- regular expressions, 87–88, 90
- remote debugging, setting up for, 176
- removing features, 214–217
- RenderDoc, 152, 170
- reports. *See* bug reports
- reproducing bugs, xix, 2, 11,
 - 15–19, 214
 - emulating the reporter, 16–17
 - frequency of, xix, 12, 69
 - networking complexity and,
 - 18–19
 - questions to ask, 14
 - re-creating the user experience,
 - 15–16
 - variations, introducing, 17–18
 - verifying fixes by, 5, 182, 185
 - steps to, 2, 12, 17–18, 27, 30–31

- restarting programs, 204
- return statement, 60, 103
- return values, 66
- rewriting functionality, 206–212
- ripgrep, 86
- roadmaps, public, 50
- root causes, xxi, xxiii, 3, 4
 - addressing, 181
 - common, 6–9
 - determining, 187–189
 - removing, 182–183
- Roslyn Analyzers, 168
- rubberducking, 175
- runtime filtering, 164
- runtime switches, 186, 211, 216–217
- Run to Cursor feature, 108
- Rust, 19, 51–52, 65, 113, 167
- Rust Playground, 167

S

- saved state, 13, 33–34
- screen recorders, 166
- screenshots, 13, 16, 142, 166
- ScreenToGif, 166
- scripting languages, 25, 31, 60
- searchable patterns in code, 90
- search engines, 47–52
 - advanced features of, 50–51
 - operators, 50–51
 - source code, searching for, 51–52
 - trying different, 49–50
- searching code, tools for, 86
- searching files, 86–91
 - search terms, 87–88
 - search tools, 86
 - configuring, 87
 - Everything, 87
 - FileLocator Pro, 86
 - fzf, 87
 - grep, 86
 - ripgrep, 86
- seed, 76
- semaphores, 75
- sentinels, 67
- Sentry, 97
- serialization, 29, 33–34
- shaders, 23
- Shadertoy, 167
- shared network directory
 - storage, 95
- sharing what you’ve learned, 186–191
- shortcuts, 38–47
- side effects, testing for, 185–186
- SIMD instructions, 102
- simultaneous events, 32
- singletons, 107
- Slack, 49, 145
- social media, 48, 49, 83, 84
- software development, 43
- source code licenses, 57
- spdlog, 162
- special characters in code, 51
- splash screens, 26
- Spotlight, 166
- sprints, 58
- SQL databases, 167
- SQL Fiddle, 167
- SSDs, 27, 87, 198
- Stack Exchange, 83
- Stack Overflow, 49, 83
- stack overflows, 106
- stakeholder consent, 217
- startup times, reducing, 26–29
- static analyzers, 167–168
- static variables, 106
- step into commands, 61
- step out commands, 61
- stepping through code, 107–108
- stopgap solution, 183
- strace, 172
- strcmp() function, 119
- Street Fighter II*, 200
- string concatenation, 51, 90
- stripped builds, 94
- structlog, 162
- structured exception handling (SEH)
 - extension, 60
- stubs, 212
- Sublime Text, 167
- suspicious systems, 43–45
- sustainable resolution, factors
 - affecting, 180
- symbol files, 94–95, 111

- symptoms, 12, 108
 - of bad data, 123–126
 - masking with workarounds, 203–205
 - tracing to root causes, 108
- System Informer, 110
- System Monitor, 173
- system tools, 172–173

T

- talking through bugs, 173–178
- Task Manager, 173
- task schedulers, 44, 46, 75
- teams, working as, 5–6
- teardown functions, 77
- technical debt, 39, 188, 201, 218, 219
- templates (C++), 22
- temporary changes, reverting, 53
- temporary objects (C++), 119
- temporary solution. *See* quick fixes, deploying
- testing, 210, 217
- test patterns, 67–68
- text editors, 86, 167
- texture compression, 23
- third-party software
 - crash reporting tools, 96–97
 - debugging, 129, 153
 - fixes in newer versions of, 56–58
 - integrating fixes from, 57–58
 - source code licenses for, 57
 - support from, 82–84
 - symbol files for, 95
- Thomas, David, 225
- threading primitives, 46
- threading-related functions, 75
- ThreadSanitizer, 169
- time-related functions, 73
- time step, 71
- time zones, 7, 132
- transformations, 126
- transient data, 125–126, 199
- try-catch block, 204

U

- UDP, 170
- UndefinedBehaviorSanitizer, 169

- underlying dependencies, 192–193
- Unified Modeling Language (UML), 142
- uninitialized memory, 68
- unity builds, 20
- Unreal Engine, 20, 58, 83
- update rates, 71
- urgency, level of, 5
- user experience (UX), 9
 - determinism and, 70
 - issues as root causes, 9
 - re-creating, 15–16
- UTF-8, 5, 89, 127
- UTF-16 and UTF-32, 127

V

- Valgrind, 171
- validation layers, 170
- variables, naming, 90
 - /var/log/ directory, 172
- verbose mode, 149
- version control history, 52–59, 193–194
- vim, 167
- virtual machines, 19–20, 25, 113, 132
 - acting like, when reading code, 113–115
- Visual Studio, 20, 25, 60, 68, 86, 111–112
- Visual Studio Code, 167
- void pointers, 22
- VsChromium, 86
- VTune, 152, 172
- Vulkan, 41, 131, 170, 215

W

- web development tools, 167, 169
- wikis, 52, 145
- Wilkes, Maurice, xix
- WinDirStat, 172
- Windows Error Reporting (WER), 96
- WinForms, 41
- WinMerge, 165

Wireshark, 170
WizTree, 172
workarounds, 203, 206
Writing Solid Code
(Maguire), 67

X

X, 49
x86 instructions, 109–110
XML, 23, 111, 124, 166

Y

Y2K bug, 132
YAML, 124
YouTube, 49

Z

zero-cost logs, 164
.zip files, 89
zlib, 28
Zstandard, 28